STARVNT VENDOR DASHBOARD V1

AUTOMATION FOUNDATION, CORE IMPLEMENTATION

& END-TO-END INTEGRATION PLAN

DAY 2
Automation Foundation + Core Module Implementation

DAY 3
End-to-End Automation Integration + Live Working Demonstration

DOCUMENT MAP

SECTION	TOPIC
01	Project Vision
02	Important Development Principle
03	Day 2 - Automation Foundation & Core Implementation
04	Yogesh - Backend Execution Plan
05	Nesar - Frontend Execution Plan
06	Day 2 Joint Checkpoint & Acceptance Criteria
07	Day 3 - End-to-End Integration
08	Yogesh - Backend Verification & Resilience
09	Nesar - Frontend Integration
10	Testing Matrix & Live Demonstration
11	Acceptance Levels
12	Optional Pending Enquiry Automation
13	Aura+ Future Handoff
14	Long-Term StarVnt Operating Architecture
15	Final Reports & Success Definition

1. PROJECT VISION

The StarVnt Vendor Dashboard must not evolve into just another dashboard where vendors log in to view data. A traditional dashboard allows a vendor to view enquiries, create quotations, manage bookings, update statuses and check notifications. The long-term StarVnt vision is significantly larger.

The platform should gradually evolve into an intelligent operational environment where business events become machine-readable signals and the system can react in a controlled, auditable and extensible way.

> **CORE VISION**
> Business happens -> Events are generated -> The system understands what happened -> Automation reacts -> Operational actions are executed.

Example - New Enquiry

A normal dashboard would simply store the enquiry and wait for the vendor to manually notice it. The StarVnt Automation-Ready Architecture should instead execute a controlled operational flow:

```
New Enquiry Created
      ↓
Business Event Generated
      ↓
Automation Engine Detects Event
      ↓
Relevant Rule Is Matched
      ↓
Conditions Are Evaluated
      ↓
Actions Are Executed
      ↓
Activity Is Created
      ↓
Notification Is Generated
      ↓
Vendor Dashboard Updates
```

> **LONG-TERM DESTINATION**
> This architecture creates the foundation for the StarVnt Business Operating System - where operational data, automation and future Aura+ intelligence work together.

2. IMPORTANT DEVELOPMENT PRINCIPLE

During Day 2 and Day 3, the team must not attempt to build a complete enterprise automation platform.

We are NOT building

✕ A Zapier-style visual automation platform

✕ A drag-and-drop workflow builder

✕ Hundreds of automation rules

✕ Complex AI-controlled autonomous workflows

✕ An automation dashboard with workflow editors

✕ A massive microservices architecture

We ARE building

✓ A clean automation architecture

✓ An event-driven foundation

✓ Rule separation

✓ Condition separation

✓ Action separation

✓ An activity system

✓ A notification system

✓ Automation logging

✓ At least one real, fully working automation flow

> **ENGINEERING PRINCIPLE**
> Simple -> Modular -> Working -> Testable -> Expandable

No unnecessary spaceship architecture. The system must actually work.

3. DAY 2 - AUTOMATION FOUNDATION & CORE MODULE IMPLEMENTATION

Day 2 Primary Mission

Build the Vendor Dashboard core business modules and create a working automation foundation. The architecture must move from documentation into real backend and frontend structure.

```
Business Module
      ↓
Business Event
      ↓
Event Dispatcher
      ↓
```

```
Automation Engine
      ↓
Rule Evaluation
      ↓
Condition Evaluation
      ↓
Action Execution
      ↓
Database Activity / Notification
```

Day 2 Priority Order

1. Working Foundation - stabilize backend, authentication, vendor identity and database connectivity.
2. Core Business Flow - implement the Enquiry module and connect it to automation.
3. Automation Foundation - implement events, rules, conditions, actions, activities and notifications.
4. Frontend Readiness - prepare the Vendor Dashboard to display automation-generated outputs.

4. YOGESH - DAY 2 BACKEND EXECUTION PLAN

Yogesh's primary responsibility is to build the backend foundation that connects Vendor business modules to a reusable automation engine.

4.1 Backend Foundation Review

```
src/
│
├── config/
├── modules/
├── middleware/
├── services/
├── models/
├── routes/
├── utils/
├── automation/
└── shared/
```

The exact folder structure may vary. What must remain stable is the separation of responsibilities.

Day 2 Foundation Check

☐ Backend Structure Ready

☐ MongoDB Connected

☐ Environment Variables Working

☐ Authentication Middleware Working

☐ JWT Verification Working

☐ Vendor Identification Working

☐ Error Handling Working

> **WHY THIS COMES FIRST**
> Automation should not be built on top of a broken foundation. If vendor identity or data ownership is unstable, automation output can be wrong and later require rework.

4.2 Enquiry Module Foundation

The first primary business entity for Vendor Dashboard automation is ENQUIRY. It represents an incoming business opportunity.

SUGGESTED FIELDS

```
enquiryId
vendorId
customerId
customerName
customerPhone
customerEmail
eventType
eventDate
eventLocation
guestCount
budget
requirements
status
createdAt
updatedAt
```

Enquiry Status Architecture

```
NEW
PENDING
RESPONDED
QUOTED
CONVERTED
CLOSED
CANCELLED
```

Minimum Enquiry APIs

METHOD / ENDPOINT	PURPOSE
POST /api/enquiries	Create a new enquiry.

GET /api/enquiries	Return only enquiries belonging to the authenticated vendor.
GET /api/enquiries/:id	Return a specific enquiry if the authenticated vendor has access.
PATCH /api/enquiries/:id/status	Update the enquiry lifecycle.

Security - Vendor Data Isolation

```
JWT
 ↓
Authentication Middleware
 ↓
Authenticated User
 ↓
Vendor Identity
 ↓
Authorization Check
 ↓
Query Only Authorized Records
```

The backend must determine vendor identity from the authenticated context and must not trust an unrestricted vendorId supplied by the frontend. This same ownership rule must later apply to quotations, bookings, notifications, activities, payments and other vendor-scoped data.

4.3 Event System Implementation

The Event System is the backbone of the automation architecture. Create a dedicated automation/events area and maintain a central event naming convention.

INITIAL BUSINESS EVENTS

```
ENQUIRY_CREATED
ENQUIRY_STATUS_CHANGED
QUOTATION_CREATED
QUOTATION_SENT
BOOKING_CREATED
BOOKING_CONFIRMED
BOOKING_CANCELLED
```

Event Payload Standard

```
{
  "eventName": "ENQUIRY_CREATED",
  "entityType": "ENQUIRY",
  "entityId": "ENQ123",
  "vendorId": "VENDOR123",
  "userId": "USER123",
  "data": {
```

```
    "status": "NEW",
    "eventType": "WEDDING",
    "vendorAssigned": true
  },
  "timestamp": "2026-09-01T10:30:00Z"
}
```

Event Dispatcher Flow

```
Create Enquiry
      ↓
Validate Request
      ↓
Save Enquiry in MongoDB
      ↓
Database Save Successful
      ↓
Emit ENQUIRY_CREATED Event
      ↓
Automation Engine Receives Event
```

> **CRITICAL EVENT RULE**
> Business operation first. Event second. Automation third. Events must be emitted only after the primary business operation succeeds.

4.4 Automation Engine

```
automation/
└── services/
    └── automation.service
```

The Automation Service is responsible for receiving an event, finding matching rules, checking whether rules are active, evaluating conditions, executing actions and recording results.

```
processEvent(event)
      ↓
Find Matching Rules
      ↓
Filter Active Rules
      ↓
Match Event Trigger
      ↓
Evaluate Conditions
      ↓
Conditions Passed?
      ↓ YES
Execute Actions
      ↓
```

Create Automation Log

Responsibility Boundary

✕ Create enquiries directly

✕ Control quotation creation

✕ Control booking business logic

✕ Replace the Enquiry Service

✕ Become a giant business controller

CORRECT SEPARATION
Business Module -> Business Service -> Business Event -> Automation Engine -> Automation Action

4.5 Rule Engine Foundation

Rule ID: NEW_ENQUIRY_RULE
Name: New Enquiry Automation
Trigger: ENQUIRY_CREATED
Status: ACTIVE
Priority: 1
Condition: Vendor Assigned = TRUE
Actions: CREATE_ACTIVITY, CREATE_NOTIFICATION

```
{
  "id": "NEW_ENQUIRY_RULE",
  "name": "New Enquiry Automation",
  "trigger": "ENQUIRY_CREATED",
  "status": "ACTIVE",
  "priority": 1,
  "conditions": [
    {
      "field": "vendorAssigned",
      "operator": "EQUALS",
      "value": true
    }
  ],
  "actions": [
    "CREATE_ACTIVITY",
    "CREATE_NOTIFICATION"
  ]
}
```

Rule Priority

VALUE	MEANING

1	HIGH
2	MEDIUM
3	LOW

The priority system is intentionally simple. It gives the platform a clean path to deterministic execution when multiple rules later match the same event.

4.6 Condition Engine

```
Event Data
      ↓
Condition
      ↓
Evaluation
      ↓
TRUE / FALSE
```

Day 2 Minimum Operators

OPERATOR	PURPOSE
EQUALS	Exact value comparison
NOT_EQUALS	Negative exact comparison
EXISTS	Field existence check

Future Operators

```
GREATER_THAN
LESS_THAN
OLDER_THAN
CONTAINS
IN_LIST
DATE_BEFORE
DATE_AFTER
HAS_NOT_RESPONDED
AMOUNT_GREATER_THAN
```

Condition Test

TEST CASE	EXPECTED RESULT

vendorAssigned = TRUE	Automation Executes
vendorAssigned = FALSE	Automation Is Skipped

4.7 Action Engine

```
executeAction()
```

The Action Engine determines which operational action should run. Day 2 requires only the minimum actions necessary to prove the architecture.

CREATE_ACTIVITY

```
activityId
vendorId
entityType
entityId
actionType
message
source
metadata
createdAt
```

```
{
  "vendorId": "VENDOR123",
  "entityType": "ENQUIRY",
  "entityId": "ENQ123",
  "actionType": "ENQUIRY_CREATED",
  "message": "New enquiry automatically recorded.",
  "source": "AUTOMATION"
}
```

CREATE_NOTIFICATION

```
notificationId
userId
vendorId
type
title
message
entityType
entityId
read
readAt
metadata
```

```
createdAt
```

```
{
  "vendorId": "VENDOR123",
  "type": "ENQUIRY",
  "title": "New Enquiry Received",
  "message": "You have received a new enquiry.",
  "entityType": "ENQUIRY",
  "entityId": "ENQ123",
  "read": false
}
```

4.8 Day 2 Main Automation Test

```
POST /api/enquiries
      ↓
Enquiry Created
      ↓
MongoDB Save Successful
      ↓
ENQUIRY_CREATED Event
      ↓
Automation Service Receives Event
      ↓
NEW_ENQUIRY_RULE Matched
      ↓
Condition Passed
      ↓
CREATE_ACTIVITY
      +
CREATE_NOTIFICATION
```

DATABASE CHECK	EXPECTED
Enquiry	CREATED
Activity	CREATED
Notification	CREATED

DAY 2 BACKEND PROOF
The automation pipeline must be observable as a real system, not merely represented as code or documentation.

5. NESAR - DAY 2 FRONTEND EXECUTION PLAN

Nesar's responsibility is to make the Vendor Dashboard automation-ready and integration-ready. He does not need to build a full automation management platform.

5.1 Vendor Dashboard Structure

```
Vendor Dashboard
│
├── Overview
├── Enquiries
├── Quotations
├── Bookings
├── Activity
└── Notifications
```

Dashboard Overview - Minimum Widgets

☐ Total Enquiries

☐ New Enquiries

☐ Pending Enquiries

☐ Active Quotations

☐ Confirmed Bookings

Mock data may be used temporarily if an API is not ready, but component interfaces and state models must be designed for real API integration. Static UI that will require a rewrite later is not acceptable.

5.2 Notification UI

```
NotificationBell
NotificationDropdown
NotificationItem
```

Required States

STATE	UI BEHAVIOUR
Empty	No new notifications.
Unread	New Enquiry Received
Read	New Enquiry Received - visually distinct read state

Notification Interaction

```
Vendor Clicks Notification Bell
      ↓
Fetch Notifications
      ↓
Display Notification List
      ↓
Vendor Clicks Notification
      ↓
Mark Notification as Read
      ↓
Navigate to Related Entity
```

Entity Navigation Contract

```
Notification
      ↓
entityType = ENQUIRY
entityId = ENQ123
      ↓
Navigate to Enquiry Details
```

5.3 Activity Timeline

```
ActivityTimeline
│
├── Icon
├── Message
├── Source
└── Timestamp
```

Example:

```
🤖 New enquiry automatically recorded.

Today • 10:32 AM
```

Activity Source Types

```
AUTOMATION
USER
SYSTEM
ADMIN
```

The component should be reusable across Customer Dashboard, Vendor Dashboard, Partner Dashboard, Admin Dashboard, CRM, Event Command Center, Booking Details and Customer Details.

5.4 API Service Layer

```
services/
│
├── api.client
├── enquiry.service
├── notification.service
└── activity.service
```

```
Dashboard Component
        ↓
Notification Service
        ↓
API Client
        ↓
Backend API
```

Do not scatter API calls directly through UI components. The service layer should centralize API access, error handling and future API contract changes.

5.5 Day 2 Frontend Success

```
Vendor Dashboard
        ↓
Notification Bell
        ↓
Notification Panel
        ↓
Activity Timeline
        ↓
Responsive Layout
```

6. DAY 2 JOINT CHECKPOINT & ACCEPTANCE

Nesar and Yogesh must jointly verify the integration contract before Day 2 closes.

6.1 Integration Contract Checks

CHECK	QUESTION
Notification created by backend	Frontend can consume response format?
Activity created by backend	Activity response structure is frontend-friendly?

Notification contains entityType + entityId	Frontend can navigate to the correct entity?

6.2 Day 2 End Reports

YOGESH - DAY 2 BACKEND REPORT

Completed:
1.
2.
3.

APIs Working:
1.
2.
3.

Events Implemented:

Rules Implemented:

Conditions Implemented:

Actions Implemented:

Database Models Created:

Automation Logs:

GitHub Repository:

Branch:

Latest Commit:

Blockers:

Day 3 Requirements:

NESAR - DAY 2 FRONTEND REPORT

Completed:
1.
2.
3.

Pages Updated:

Components Created:

API Integration Status:

Responsive Status:

GitHub Repository:

Branch:

Latest Commit:

Blockers:

Day 3 Requirements:

6.3 Day 2 Final Acceptance Criteria

✓ Enquiry Module Exists

✓ Vendor Authorization Works

✓ Business Event Exists

✓ Event Dispatcher Works

✓ Automation Service Exists

✓ Automation Rule Exists

✓ Condition Evaluation Exists

✓ CREATE_ACTIVITY Works

✓ CREATE_NOTIFICATION Works

✓ Activity Is Stored

✓ Notification Is Stored

✓ Frontend Is Ready to Display Outputs

7. DAY 3 - END-TO-END AUTOMATION INTEGRATION & LIVE WORKING DEMONSTRATION

7.1 Day 3 Primary Mission

Day 3 is not an architecture discussion day. It is Integration Day. The goal is to connect the individual modules and prove that a real automation flow works from frontend to backend and back to the frontend.

```
VENDOR LOGIN
      ↓
JWT VERIFIED
      ↓
VENDOR DASHBOARD
```

```
      ↓
CREATE ENQUIRY
      ↓
BACKEND DATABASE SAVE
      ↓
BUSINESS EVENT
      ↓
AUTOMATION ENGINE
      ↓
RULE MATCH
      ↓
CONDITION CHECK
      ↓
ACTION EXECUTION
      ↓
ACTIVITY SAVED
      +
NOTIFICATION SAVED
      ↓
FRONTEND FETCH
      ↓
VENDOR SEES UPDATE
```

DAY 3 GOAL
Move from architecture to proof: a real user action must cause a real automated operational response that is visible in the Vendor Dashboard.

8. YOGESH - DAY 3 BACKEND VERIFICATION & RESILIENCE

8.1 End-to-End API Verification

API	EXPECTED
POST Login	Valid authentication response, JWT generated, authenticated user returned
POST Create Enquiry	Enquiry stored successfully
GET Vendor Enquiries	Only authorized vendor data returned
GET Single Enquiry	Access controlled by vendor ownership
PATCH Update Enquiry Status	Lifecycle state updates correctly
GET Activities	Only authorized vendor activities returned

GET Notifications	Only authorized vendor notifications returned
PATCH Mark Notification Read	Notification read state updates

8.2 Vendor Authorization Test

```
Vendor A
  ↓
Creates ENQUIRY A

Vendor B
  ↓
Attempts to access ENQUIRY A

Expected:
FORBIDDEN or NOT FOUND
```

The same isolation test must apply to notifications and activities, and later to quotations, bookings and other vendor-owned records.

8.3 Automation Failure Handling

```
Enquiry Created Successfully
      ↓
Event Triggered
      ↓
Activity Created Successfully
      ↓
Notification Creation Fails
```

The primary enquiry should remain created. A failure in an automation side-effect should be recorded without corrupting the core business operation.

Automation Result Statuses

STATUS	MEANING
SUCCESS	All expected actions completed.
FAILED	Automation could not complete the intended operation.
PARTIAL_FAILURE	Some actions completed and at least one action failed.

8.4 Automation Logging

```
automationLogId
eventName
ruleId
entityType
entityId
vendorId
status
actionsExecuted
failedActions
error
createdAt
```

```
Event: ENQUIRY_CREATED
Rule: NEW_ENQUIRY_RULE
Status: SUCCESS
```

Automation logs will support future debugging, monitoring, analytics, failure detection and administrative auditing.

9. NESAR - DAY 3 FRONTEND INTEGRATION

9.1 Replace Mock Data with Real APIs

```
Mock Notifications
      ↓
Real Notification API

Mock Activities
      ↓
Real Activity API
```

9.2 Enquiry Creation Flow

```
Vendor Opens Enquiry Form
      ↓
Enters Required Data
      ↓
Clicks Submit
      ↓
POST /api/enquiries
      ↓
Loading State
      ↓
Success Response
      ↓
```

> Enquiry Created Successfully

After Successful Creation

☐ Refresh Enquiry List

☐ Refresh Notification Count

☐ Refresh Notifications

☐ Refresh Activity Timeline

9.3 Automation Output UI

> 🤖 AUTOMATION
> New enquiry automatically recorded.
>
> 🔔 New Enquiry Received
> You have received a new enquiry.

9.4 Integration Strategy

```
Create Enquiry
      ↓
API Success
      ↓
Refetch Enquiries
      ↓
Refetch Notifications
      ↓
Refetch Activities
      ↓
Update Dashboard
```

Real-time WebSockets are not mandatory for Day 3. Simple API refetching is sufficient. Future upgrades may introduce WebSockets, Server-Sent Events and push notifications.

10. TESTING MATRIX & LIVE DEMONSTRATION

10.1 Day 3 Complete Testing Matrix

TEST	INPUT	EXPECTED RESULT
1 - Login	Valid vendor credentials	Dashboard access

2 - Invalid Login	Wrong password	Clear error message
3 - Create Enquiry	Valid enquiry data	Enquiry saved successfully
4 - Event	Create enquiry	ENQUIRY_CREATED emitted
5 - Rule Matching	Event = ENQUIRY_CREATED	NEW_ENQUIRY_RULE matched
6 - Condition	vendorAssigned = TRUE	Condition passed
7 - Action	Rule passed	Activity + notification created
8 - Frontend Output	Refetch data	Notification + activity visible
9 - Mark as Read	Click notification	read = true
10 - Data Isolation	Vendor A vs Vendor B	Unauthorized data is inaccessible

10.2 Day 3 Final Live Demonstration

Screenshots alone are not enough. The team should demonstrate the complete real workflow in sequence.

1. Login as Vendor and show successful authentication.
2. Open the Vendor Dashboard and show current notifications and activities.
3. Show the current notification count.
4. Create a new enquiry with realistic test data.
5. Show successful enquiry creation.
6. Verify backend/API response and correct vendor ownership.
7. Show the automation-created activity.
8. Show the automation-created notification.
9. Refresh or refetch the dashboard.
10. Demonstrate the final state: enquiry exists, activity exists and notification exists.

```
Vendor Dashboard
      ↓
New Enquiry Exists
      ↓
Automation Activity Exists
      ↓
Notification Exists
```

11. DAY 3 ACCEPTANCE LEVELS

LEVEL 1 - CODE STRUCTURE

Folders, files and modules exist.

✕ Not enough.

LEVEL 2 - MODULES WORK

Enquiry, Notification and Activity functions work.

✓ Good foundation.

LEVEL 3 - AUTOMATION WORKS

```
Event
   ↓
Rule
   ↓
Condition
   ↓
Action
```

✓ Strong proof of an event-driven architecture.

LEVEL 4 - COMPLETE STARVNT FLOW

```
Real User
   ↓
Frontend
   ↓
Backend
   ↓
Database
   ↓
Business Event
   ↓
Automation
   ↓
Activity / Notification
   ↓
Frontend Output
```

> **ACTUAL DAY 3 GOAL**
> Level 4 - a real user action should produce a real automated result visible in the product.

12. OPTIONAL DAY 3 BONUS - PENDING ENQUIRY REMINDER

Implement this only after the main ENQUIRY_CREATED automation is stable.

```
Scheduled Job
      ↓
Find Pending Enquiries
      ↓
Check Created Time
      ↓
48 Hours Passed?
      ↓ YES
Trigger Automation
      ↓
CREATE_ALERT
      ↓
CREATE_NOTIFICATION
```

⚠️ Follow-up Required

This enquiry has been pending for more than 48 hours.

Duplicate Reminder Prevention

The same enquiry must not generate the same reminder repeatedly.

```
Option A:
reminderSent = true

Option B:
Check Automation Log
      ↓
Already Reminded?
      ↓ YES
Skip
```

RULE
Prevent notification spam. Automation must be safe to run repeatedly without creating uncontrolled duplicate actions.

13. AURA+ FUTURE HANDOFF CONTRACT

Aura+ does not need to be built during Day 2 or Day 3. However, the Automation Engine should be structured so future Aura+ recommendations can safely become operational actions.

```
{
  "source": "AURA_PLUS",
  "recommendationType": "FOLLOW_UP",
  "entityType": "ENQUIRY",
  "entityId": "ENQ123",
  "confidence": 0.91
}
```

```
Aura+ Intelligence
      ↓
Detects Business Pattern
      ↓
Creates Recommendation
      ↓
Automation Engine Receives Recommendation
      ↓
Rule Evaluated
      ↓
Action Executed
```

Potential future actions include CREATE_ALERT, CREATE_TASK and CREATE_FOLLOW_UP.

Future Example - Vendor Response Performance

```
Vendor A

Total Enquiries: 20
Responded: 9
Average Response Time: 14 Hours
Conversion: Low
```

Aura+ may identify a pattern indicating that slow response time is affecting conversion and recommend a follow-up within two hours. The future operational chain becomes:

```
Aura+ Recommendation
      ↓
Automation Engine
      ↓
CREATE_ALERT
      ↓
CREATE_TASK
      ↓
CREATE_FOLLOW_UP
```

> ⚠️ Response Performance Alert
>
> Your enquiry response time is currently above the recommended threshold.

14. LONG-TERM STARVNT OPERATING ARCHITECTURE

```
CUSTOMERS
VENDORS
PARTNERS
EMPLOYEES
ADMINS
      ↓
STARVNT CORE
      ↓
BUSINESS DATA
      ↓
AUTOMATION ENGINE
      ↓
OPERATIONAL ACTION
      ↓
AURA+
      ↓
ANALYSIS + INTELLIGENCE
      ↓
RECOMMENDATIONS
      ↓
AUTOMATION AGAIN
```

14.1 The StarVnt Intelligence Loop

```
BUSINESS HAPPENS
      ↓
DATA IS CREATED
      ↓
AURA+ ANALYZES
      ↓
INSIGHT GENERATED
      ↓
RECOMMENDATION CREATED
      ↓
AUTOMATION EXECUTES
      ↓
OPERATIONAL ACTION
      ↓
BUSINESS RESULT
      ↓
NEW DATA
      ↓
```

AURA+ ANALYZES AGAIN

SYSTEM IDEA
Business events provide the operational signals. Automation provides controlled execution. Aura+ provides future intelligence and recommendations. These layers should remain distinct but interoperable.

15. FINAL REPORTS & SUCCESS DEFINITION

15.1 Day 3 Final Report Template

🚀 STARVNT VENDOR DASHBOARD V1
DAY 3 FINAL REPORT

==========

AUTHENTICATION

Login Working:
YES / NO

JWT Working:
YES / NO

Vendor Authorization:
YES / NO

==========

ENQUIRY MODULE

Create:
YES / NO

List:
YES / NO

Get Single:
YES / NO

Status Update:
YES / NO

==========

AUTOMATION

Event: ENQUIRY_CREATED
Working: YES / NO

Rule: NEW_ENQUIRY_RULE
Working: YES / NO

Condition Evaluation:
YES / NO

==========

ACTIONS

CREATE_ACTIVITY:
YES / NO

CREATE_NOTIFICATION:
YES / NO

==========

DATABASE

Enquiry Stored:
YES / NO

Activity Stored:
YES / NO

Notification Stored:
YES / NO

Automation Log Stored:
YES / NO

==========

FRONTEND

Notification Display:
YES / NO

Activity Timeline:
YES / NO

Real API Integration:
YES / NO

Responsive UI:
YES / NO

==========

END-TO-END DEMO

Vendor Login
↓
Create Enquiry
↓
Automation Trigger
↓

```
Activity Created
↓
Notification Created
↓
Dashboard Updated

STATUS:
WORKING / PARTIAL / NOT WORKING

================================

GITHUB

Repository:

Branch:

Latest Commit:

================================

BLOCKERS

1.

2.

================================

DAY 4 RECOMMENDATION

1.

2.

================================
```

15.2 Final Team Instruction

DAY 2 = BUILD THE FOUNDATION.

Focus on the Enquiry Module, Events, Automation Engine, Rules, Conditions, Actions, Activities and Notifications.

DAY 3 = PROVE THE COMPLETE SYSTEM WORKS.

Focus on the complete chain from frontend to backend to database to business event to automation to activity/notification and back to the frontend.

15.3 The Most Important Success Question

At the end of Day 3, the question is not how many folders were created, how many lines of code were written or how many pages of documentation were produced.

THE REAL QUESTION
Can a real business action automatically trigger a meaningful operational response?

```
NEW ENQUIRY
      ↓
EVENT
      ↓
AUTOMATION
      ↓
ACTIVITY
      +
NOTIFICATION
      ↓
VENDOR DASHBOARD
```

DAY 3 SUCCESS
If this flow works end-to-end with real frontend, backend, database and API interaction, the automation foundation is proven.

STARVNT BUSINESS OPERATING SYSTEM

At that point, StarVnt is no longer simply building a Vendor Dashboard. It is laying the first architectural brick of a future event-driven business infrastructure.

```
Business happens.
Events are created.
Automation reacts.
Operations move forward.
Aura+ analyzes patterns.
StarVnt becomes smarter over time.
```

STARVNT

Business happens. The system understands. Automation reacts. Aura+ thinks ahead. The ecosystem evolves.